

✅ Problem 1: Exception Propagation Puzzle

Problem Statement:

Write a Java program where the try, catch, and finally blocks each throw a different RuntimeException. Observe which exception gets propagated and why.

Objective: Understand how exceptions are handled when multiple blocks throw.

✅ Problem 2: Convert Manual Resource Management to Try-With-Resources

Problem Statement:

You are given a program that reads a file using BufferedReader and handles closing manually in the finally block.

Task: Rewrite the code using try-with-resources to handle resource management automatically.

Objective: Practice resource handling and cleanup using modern Java features.

✅ Problem 3: Suppressed Exceptions with Try-With-Resources

Problem Statement:

Create a custom class implementing AutoCloseable whose close() method throws an exception.

Inside a try-with-resources block, also throw an exception from within the try block.

Task: Capture the primary and suppressed exceptions, and print both.

Objective: Learn how Java handles multiple exceptions (suppressed and primary).

✅ Problem 4: Use Multi-Catch Block

Problem Statement:

You are writing a program that might throw either IOException or SQLException from a method.

Task: Catch both exceptions in a single catch block using multi-catch syntax.

Objective: Avoid redundant code using Java's multi-catch feature.

✅ Problem 5: Create and Use a Custom Checked Exception

Problem Statement:

Create a custom checked exception called InvalidAgeException.

Write a method validateAge(int age) that throws this exception if age is less than 18.

Task: Call this method from main() and handle the exception appropriately.

Objective: Learn to create, throw, and handle user-defined checked exceptions.

 Problem 6: Rethrow an Exception After Logging

Problem Statement:

Create a method that reads from a file.

If a FileNotFoundException is caught, log it using printStackTrace(), then rethrow it to be handled in main().

Objective: Learn how to rethrow exceptions after partial handling (logging, auditing).

 Problem 7: Predict Output of Nested Try-Catch-Finally

Problem Statement:

Write a nested try-catch-finally block that throws and catches an ArithmeticException.

Task: Print statements in every block and determine the output sequence.

Objective: Understand execution flow with nested exception blocks.